

KENYA INSTITUTE OF CURRICULUM DEVELOPMENT (KICD) —
CBC ALIGNED

GRADE 10 COMPUTER SCIENCE

Senior School · Science, Technology, Engineering &
Mathematics (STEM) Pathway

LEARNER'S NOTES

Comprehensive study notes covering Foundation of Computer Science, Computer Networking, and Software Development — including idea boxes, reflection questions, case studies, worksheets, exercises, end-of-topic questions, and a full answers chapter.

Based on the KICD Grade 10 Computer Science Senior School Curriculum Design (June 2024).
Prepared for learners aged 14–16 years.

About These Notes

THESE ARE LEARNER'S NOTES. This book has been developed to support learners aged 14–16 years (Grade 10) studying Computer Science under Kenya's Competency Based Curriculum (CBC) Senior School programme. It is organised strictly according to the three strands of the official KICD Grade 10 Computer Science Curriculum Design: **Foundation of Computer Science, Computer Networking, and Software Development.**

Each topic in this book follows the same structure to help you learn effectively:

Learning Outcomes What you should be able to do after studying the topic.

Notes The main explanation of concepts, with tables, examples and definitions.

Idea Box A quick tip or memory aid to help you remember a key idea.

Case Study A real-life scenario to help you apply what you have learnt.

REFLECT AND DISCUSS Open questions with no single fixed answer, for you to think about or discuss with classmates.

Worksheet A hands-on activity or table to complete, individually or in groups.

Exercise Short questions to test your understanding as you go.

End of Topic Questions Revision questions to prepare you for assessment. Answers are provided in the Answers chapter at the end of the book.

Table of Contents

STRAND 1.0: Foundation of Computer Science

1.1 Evolution and Development of Computers	6 lessons
1.2 Computer Organisation and Architecture	4 lessons
1.3 Input/Output (I/O) Devices	12 lessons
1.4 Computer Storage	6 lessons
1.5 Central Processing Unit (CPU)	6 lessons
1.6 Operating System (OS)	12 lessons
1.7 Computer Setup	10 lessons

STRAND 2.0: Computer Networking

2.1 Data Communication	6 lessons
------------------------	-----------

2.2 Data Transmission Media	8 lessons
2.3 Computer Network Elements	10 lessons
2.4 Network Topologies	8 lessons

STRAND 3.0: Software Development

3.1 Computer Programming Concepts	14 lessons
3.2 Program Development	15 lessons
3.3 Identifiers and Operators	18 lessons
3.4 Control Structures	17 lessons
3.5 Data Structures (Containers)	14 lessons
3.6 Functions	14 lessons

ANSWERS TO END OF TOPIC QUESTIONS

APPENDIX: ASSESSMENT METHODS & LEARNING RESOURCES

1.0

Foundation of Computer Science

This strand introduces you to the building blocks of computers - from their historical evolution to the hardware components (input/output devices, storage, the CPU) and the operating systems that make them usable. By the end of this strand, you will understand how a computer is built, how it processes data, and how to set one up safely for use.

1.1 Evolution and Development of Computers

Suggested duration: 6 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Identify early computing devices and how they relate to the evolution of electronic computers.
- Describe the principal technologies that defined the development of computers.
- Relate the principal technologies to their respective computer generations.
- Appreciate the technological advancement in the development of computers.

Key Terms: Abacus, Analytical Engine, Vacuum tube, Transistor, Integrated Circuit (IC), Generation of computers, Microprocessor

A computer is an electronic device that accepts data, processes it and produces information as output. Before electronic computers existed, human beings used simple mechanical devices to help them count and calculate. Understanding how these devices developed into the powerful computers we use today helps us appreciate how far technology has come, and gives us clues about where it is going next.

1.1.1 Early Computing Devices

Long before electricity was harnessed for computing, several devices were invented to make counting and calculation easier:

Device	Approx. Period	Description
Abacus	Ancient (c. 2400 BC)	A frame with beads sliding on rods, used for addition and subtraction. Considered the earliest counting device.
Napier's Bones	1617	A set of numbered rods invented by John Napier used to perform multiplication and division through simple rod alignment.
Pascaline	1642	A mechanical adding machine invented by Blaise Pascal that used gears to add and subtract numbers automatically.
Slide Rule	1622	A device with sliding rulers used to perform multiplication, division and other calculations using logarithms.
Jacquard Loom	1801	A weaving loom controlled by punched cards. It introduced the idea of using cards to store and feed instructions into a machine - a forerunner of computer programming.
Difference Engine	1822	Designed by Charles Babbage to automatically compute mathematical tables using the method of finite differences.

Analytical Engine	1837	Also designed by Charles Babbage, this device introduced concepts found in modern computers: an input unit (punched cards), a processing unit (the "mill"), a memory unit (the "store") and an output unit. Ada Lovelace wrote instructions for it, earning her the title of the world's first computer programmer.
--------------------------	------	---

1.1.2 Principal Technologies in Computer Development

The transition from mechanical devices to modern computers was driven by a series of key electronic technologies:

- **Vacuum Tubes** - Glass tubes that controlled electric current and could act as switches or amplifiers. They were large, generated a lot of heat and consumed a lot of power.
- **Transistors** - Small semiconductor devices that performed the same switching function as vacuum tubes but were smaller, faster, cheaper and more reliable, and produced far less heat.
- **Integrated Circuits (IC)** - Many transistors and other components combined onto a single small chip of silicon, drastically reducing the size of computers while increasing their speed.
- **Large Scale Integration (LSI)** - Thousands of transistors packed onto a single chip.
- **Very Large Scale Integration (VLSI)** - Hundreds of thousands to millions of transistors on a single chip, leading to the microprocessor - the "brain" found in personal computers, phones and other devices.

1.1.3 Generations of Computers

Generation	Period	Principal Technology	Key Features
First	1940-1956	Vacuum tubes	Huge in size, used a lot of electricity, generated much heat, e.g. ENIAC.
Second	1956-1963	Transistors	Smaller, faster, more reliable and energy efficient than first generation computers.
Third	1964-1971	Integrated Circuits	Even smaller and faster; introduction of keyboards and monitors for input/output.
Fourth	1971-present	Microprocessors (LSI/VLSI)	Personal computers became possible; introduction of graphical user interfaces (GUI) and networking.
Fifth	Present and beyond	Artificial Intelligence, VLSI, parallel processing	Devices that can "think", learn and respond intelligently, e.g. voice assistants and robotics.

IDEA BOX

Each new generation of computers has generally been **smaller, faster, cheaper and more powerful** than the one before it. This trend of continuously shrinking components while increasing computing power is often referred to informally as following "*Moore's Law*" - the observation that the number of transistors on a chip roughly doubles every two years.

CASE STUDY: From Punched Cards to Smartphones

In 1943, a computer called the ENIAC filled an entire room, weighed about 30 tonnes, and needed a team of engineers to operate it. It could perform about 5,000 additions per second. Today, a simple smartphone that fits in your pocket can perform billions of calculations per second and can also make calls, browse the internet, take photographs and run thousands of applications. Discuss with a partner how the shrinking of computer components (from vacuum tubes to microprocessors) has changed the way people live and work in Kenya today.

REFLECT AND DISCUSS

1. Why do you think Charles Babbage is referred to as the 'Father of the Computer' even though he never built a complete working machine?
2. In what ways would your life be different if computers were still the size of a room, as they were in the first generation?

Worksheet 1.1: Matching Generations to Technologies

Copy and complete the table below in your notebook.

Generation	Principal Technology	One Key Feature

EXERCISE 1.1

1. List any four ancient or medieval computing devices in the order they were invented.
2. Explain the contribution of the Jacquard loom to the development of modern computers.
3. Draw a timeline showing the five generations of computers and their principal technologies.
4. Explain why Integrated Circuits (ICs) were a major improvement over transistors.

END OF TOPIC QUESTIONS 1.1

1. Which ancient device is considered the earliest counting device?
A) Pascaline B) Abacus C) Slide rule D) Napier's Bones
2. Name the person who designed the Analytical Engine.
3. State the principal technology used in third generation computers.
4. Give two characteristics of first generation computers.
5. Explain the relationship between Very Large Scale Integration (VLSI) and the microprocessor.

(See the Answers chapter at the end of this book to check your answers.)

1.2 Computer Organisation and Architecture

Suggested duration: 4 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Describe the functional organisation and architecture of a von Neumann computer.
- Analyse the relationships among the functional elements of a von Neumann computer.
- Create a model of a computer architecture depicting its structural elements.
- Use binary, octal and hexadecimal number systems to represent data.
- Appreciate the importance of computer architecture in computing.

Key Terms: Von Neumann architecture, RISC, CISC, Bus, Fetch-execute cycle, Binary, Hexadecimal

Computer architecture describes how the different parts of a computer are organised and how they work together to process data. Understanding architecture helps us know why some computers are faster or more powerful than others.

1.2.1 The Von Neumann Architecture

Most modern computers are based on a design proposed by John von Neumann. This architecture has four main functional elements:

- **Input Unit** - accepts data and instructions from the outside world (e.g. keyboard, mouse).
- **Central Processing Unit (CPU)** - processes data. It contains the Arithmetic and Logic Unit (ALU) and Control Unit (CU).
- **Memory (Storage) Unit** - stores data and instructions, both those waiting to be processed and the results of processing.
- **Output Unit** - presents processed data (information) to the user (e.g. monitor, printer).

A key idea of the von Neumann architecture is the **stored-program concept**: both the instructions (the program) and the data they work on are stored together in the same memory. Buses (electronic pathways) connect these elements, allowing data, addresses and control signals to move between them.

1.2.2 RISC vs CISC

Feature	RISC (Reduced Instruction Set Computer)	CISC (Complex Instruction Set Computer)
Instruction set	Small, simple instructions	Large, complex instructions
		Instructions may take several clock cycles

Execution speed	Each instruction executes in one clock cycle	
Examples	ARM processors (used in most smartphones)	Intel and AMD x86 processors (used in most desktop computers)

1.2.3 The Fetch-Execute Cycle

The CPU processes instructions one at a time using a repeating cycle:

1. **Fetch** - the next instruction is retrieved from memory.
2. **Decode** - the control unit interprets what the instruction means.
3. **Execute** - the instruction is carried out (e.g. an addition is performed by the ALU).

This cycle repeats, at extremely high speed, for every instruction a program contains.

1.2.4 Number Systems

Computers represent all data - numbers, letters, images and sound - using the **binary number system** (base 2), which uses only the digits 0 and 1. Because long strings of binary digits are hard for humans to read, we often use **octal** (base 8) and **hexadecimal** (base 16) as shorthand.

System	Base	Digits Used
Binary	2	0, 1
Octal	8	0 - 7
Decimal	10	0 - 9
Hexadecimal	16	0 - 9, A, B, C, D, E, F

Example: Convert 25 (decimal) to binary.

$$25 \div 2 = 12 \text{ remainder } 1$$

$$12 \div 2 = 6 \text{ remainder } 0$$

$$6 \div 2 = 3 \text{ remainder } 0$$

$$3 \div 2 = 1 \text{ remainder } 1$$

$$1 \div 2 = 0 \text{ remainder } 1$$

Reading the remainders from bottom to top: **25 (decimal) = 11001 (binary)**

IDEA BOX

A single binary digit (0 or 1) is called a **bit**. A group of 8 bits is called a **byte**, which is enough to represent one character, such as a letter or a number.

CASE STUDY: Choosing the Right Processor

A school is setting up a computer laboratory. The ICT technician recommends laptops with ARM-based processors for a set of tablets used for reading e-books (long battery life, simple tasks), and desktop computers with Intel CISC processors for the computer lab used for graphic design and video editing (more processing power for complex tasks). Discuss why different processor architectures suit different purposes.

REFLECT AND DISCUSS

1. Why do you think it is important for a computer's data and instructions to be stored in the same memory?
2. Explain, in your own words, why binary is the most fundamental number system in computing.

Worksheet 1.2: Number System Conversions

Convert the following decimal numbers to binary, octal and hexadecimal, showing your working: 18, 45, 100, 63, 250.

Decimal	Binary	Octal	Hexadecimal

EXERCISE 1.2

1. Name the four functional elements of a von Neumann computer.
2. Describe the role of the bus in a computer architecture.
3. Differentiate between RISC and CISC architecture.
4. Convert 39 (decimal) to binary, showing your working.
5. Explain the three stages of the fetch-execute cycle.

END OF TOPIC QUESTIONS 1.2

1. The idea that a program's instructions and data are stored in the same memory is called the _____ concept.
2. Which of the following is an example of a RISC processor?
A) Intel Core i7 B) AMD Ryzen C) ARM processor D) Pentium 4
3. List the three stages of the fetch-execute cycle in order.
4. Convert the binary number 1011 to decimal.
5. Explain two differences between RISC and CISC architectures.

(See the Answers chapter at the end of this book to check your answers.)

1.3 Input/Output (I/O) Devices

Suggested duration: 12 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Describe types of input and output devices used in computer systems.
- Examine criteria used in selecting input and output devices.
- Use input and output devices to perform tasks.
- Appreciate the advancement of input and output devices used in computer systems.

Key Terms: Input device, Output device, Scanner, QR code, Plotter, Actuator, Digitizer

1.3.1 Input Devices

Input devices allow us to enter data and instructions into a computer. They are grouped by how they capture data:

Category	Examples
Keying devices	Keyboard, keypad
Pointing devices	Mouse, trackball, joystick, touchpad
Scanning devices	Barcode scanner, image scanner, 2D/3D scanners
Voice input devices	Microphone
Touch screens	Resistive, capacitive and infra-red touch screens
Other	Digitizer, digital camera, Quick Response (QR) code reader

1.3.2 Output Devices

Output devices present processed data (information) to the user in a usable form:

- **Monitors** - display visual output on a screen.
- **Printers** - produce output on paper (hard copy).
- **Speakers** - produce audio output.
- **Projectors** - display output on a large screen or wall.
- **Plotters** - produce high-quality line drawings, often used for architectural and engineering designs.
- **Actuators** - convert electronic signals into physical motion, e.g. in robotics.

1.3.3 Selecting Input and Output Devices

When choosing an input or output device, consider:

- **Purpose/Task** - what will the device be used for?
- **Speed** - how quickly does data need to be entered/output?
- **Cost** - is the device affordable?
- **Compatibility** - will the device work with the existing computer system?
- **Ease of use** - is the device user-friendly?
- **Accuracy** - does the device capture or produce data correctly?

1.3.4 QR Codes and Barcodes

A **barcode** stores data as a series of parallel lines of varying widths, read in one direction (1-dimensional). A **QR (Quick Response) code** stores data in a two-dimensional pattern of squares, and can hold much more information than a barcode, including text, URLs and contact details.

Barcode	QR Code
1-dimensional	2-dimensional
Stores less data (usually numbers)	Stores more data (text, links, numbers)
Needs a laser scanner	Can be scanned using a smartphone camera

IDEA BOX

Some devices, like touch screens, can act as **both input and output devices** - they display information (output) and also accept touch commands (input).

CASE STUDY: Digitising a School Library

A school library wants to speed up the process of borrowing and returning books. The librarian decides to attach a barcode to every book and issue each student a card with a QR code containing their admission number. When a student borrows a book, the librarian scans both codes with a barcode scanner connected to a computer, and the system automatically records the transaction. Discuss how this use of input devices improves the efficiency and accuracy of the library compared to a manual record book.

REFLECT AND DISCUSS

1. Which input and output devices have you interacted with today? Group them as input, output, or both.
2. Why might a hospital choose a touch screen over a keyboard and mouse for a nurse's information kiosk?

Worksheet 1.3: Classifying I/O Devices

In groups, list 10 devices found in your school's computer lab or ICT room and classify each as an input device, an output device, or both. Then identify one factor that influenced its selection.

Device	Input/Output/Both	Reason for selection

EXERCISE 1.3

1. Give three examples each of keying devices and pointing devices.
2. Differentiate between a barcode and a QR code.
3. State four factors to consider when selecting an input or output device.
4. Explain the function of a plotter and give one situation where it would be preferred over a printer.

END OF TOPIC QUESTIONS 1.3

1. Which of the following is a pointing device?
A) Keyboard B) Scanner C) Mouse D) Microphone
2. State one advantage of a QR code over a traditional barcode.
3. Name a device that can function as both an input and output device.
4. List any four factors to consider when acquiring an input or output device.
5. Explain how the use of input and output devices can improve efficiency in a school library, using an example.

(See the Answers chapter at the end of this book to check your answers.)

1.4 Computer Storage

Suggested duration: 6 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Identify types of storage used in computer systems.
- Categorise types of storage used in computer systems.
- Read data from and write data to a computer storage.
- Establish criteria used to select computer storage.
- Acknowledge safety of data in computer storage media.

Key Terms: Primary memory, RAM, ROM, Secondary storage, SSD, HDD, Remote storage

1.4.1 Categories of Computer Storage

Computer storage is broadly categorised into **primary (main) storage** and **secondary (auxiliary) storage**.

(a) Primary Memory

Type	Description
RAM (Random Access Memory)	Temporary, volatile memory that holds data and instructions currently being used by the CPU. Its contents are lost when power is switched off.
ROM (Read Only Memory)	Permanent, non-volatile memory that stores instructions needed to start up the computer (e.g. BIOS). Its contents cannot easily be changed.

RAM technology is further divided into **DRAM** (Dynamic RAM - needs to be refreshed constantly, cheaper, used as main memory) and **SRAM** (Static RAM - does not need refreshing, faster but more expensive, used in cache memory).

(b) Secondary Storage

Internal Secondary Storage	External Secondary Storage
Hard Disk Drive (HDD)	DVD/CD and DVD-RAM
Solid State Drive (SSD)	Blu-ray disc
	USB memory stick / flash memory
	Removable hard drive

1.4.2 Comparing HDD and SSD

HDD	SSD
Uses spinning magnetic disks	Uses flash memory chips (no moving parts)
Slower read/write speed	Faster read/write speed
Cheaper per gigabyte	More expensive per gigabyte
More prone to mechanical failure	More durable, resistant to shocks

1.4.3 Remote Storage

Remote storage (cloud storage) allows data to be stored on servers accessed over the internet rather than on a local device, e.g. Google Drive, Dropbox, OneDrive. It allows access to data from anywhere and acts as a backup in case a local device is lost or damaged, but it depends on an internet connection and raises questions about data privacy and security.

1.4.4 Selecting Computer Storage

Consider: **capacity** (how much data it can hold), **speed** (read/write speed), **portability** (can it be carried around), **durability/robustness**, **cost**, and **data security**.

IDEA BOX

Remember: RAM is *volatile* (loses data without power); ROM and secondary storage devices are *non-volatile* (retain data even without power). Always save your work frequently to secondary storage to avoid losing it!

CASE STUDY: Backing Up School Records

A school stores all its student records on a single computer's hard disk. One day, the hard disk fails and all records are lost, since there was no backup. After this incident, the school ICT committee decides to keep copies of all records on an external hard drive and on a cloud storage service. Explain, using the ideas of local versus remote storage, how this new approach protects the school's data.

REFLECT AND DISCUSS

1. Why do computer manufacturers include both RAM and ROM in a computer instead of just one type of memory?
2. What safety precautions should you take when handling storage devices such as flash disks or external hard drives?

Worksheet 1.4: Comparing Storage Devices

Research and compare an HDD, an SSD, and a USB flash drive on: capacity, speed, portability, durability, and cost.

Storage Device	Capacity	Speed	Portability	Durability	Cost

EXERCISE 1.4

1. Differentiate between primary and secondary storage.
2. State two differences between RAM and ROM.
3. Explain the difference between DRAM and SRAM.
4. Give three examples of external secondary storage.
5. Explain the meaning of remote storage and give two examples.

END OF TOPIC QUESTIONS 1.4

1. Which type of memory is volatile?
A) ROM B) RAM C) HDD D) SSD
2. Give one example of an internal secondary storage device.
3. State one advantage of an SSD over an HDD.
4. Explain what is meant by 'remote storage' and give one example.
5. Discuss three criteria you would use to select a suitable storage device for keeping a school's examination records.

(See the Answers chapter at the end of this book to check your answers.)

1.5 Central Processing Unit (CPU)

Suggested duration: 6 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Describe structural elements of the CPU of a computer system.
- Relate structural elements of the CPU to their functions.
- Examine types of CPUs in computing devices.
- Appreciate the role of the CPU in computing.

Key Terms: ALU, Control Unit, Registers, Bus, Core, Clock speed

The Central Processing Unit (CPU) is often called the "brain" of the computer because it carries out instructions and controls the operations of all other parts.

1.5.1 Structural Elements of the CPU

Element	Function
Arithmetic and Logic Unit (ALU)	Performs arithmetic operations (addition, subtraction, multiplication, division) and logical operations (comparisons such as greater than, equal to).
Control Unit (CU)	Directs and coordinates the activities of the entire computer system; fetches, decodes and manages execution of instructions.
Registers	Small, extremely fast storage locations within the CPU that hold data currently being processed, memory addresses, or intermediate results.
Buses	Electronic pathways that carry data, addresses and control signals between the CPU and other components.

1.5.2 The Fetch-Decode-Execute Cycle Revisited

Working with memory and registers, the CPU repeats the fetch-decode-execute cycle for every single instruction in a program - millions or billions of times per second.

1.5.3 Types of CPUs

CPUs can be compared according to:

- **Instruction set:** RISC vs CISC (see Topic 1.2).

- **Word length:** 16-bit, 32-bit, 64-bit - the amount of data the CPU can process at once. A 64-bit CPU can process more data per cycle than a 32-bit CPU.
- **Core design:** single-core, dual-core, quad-core, hexa-core, octa-core, deca-core. A core is an independent processing unit within the CPU. More cores allow a computer to perform more tasks simultaneously (multitasking).

1.5.4 Factors to Consider When Selecting a CPU

- Clock speed (measured in GHz) - how many cycles the CPU can perform per second.
- Number of cores - for multitasking and running demanding applications.
- Cache size - fast memory built into the CPU for frequently used data.
- Compatibility with the motherboard.
- Power consumption and heat generation.
- Cost.

IDEA BOX

The CPU can be compared to the human brain: the ALU is like the part of the brain that solves problems and makes calculations, while the Control Unit is like the part that coordinates and directs actions - deciding what to do and when.

CASE STUDY: Choosing a CPU for Different Tasks

James wants to buy a laptop mainly for typing documents, browsing the internet and watching videos. His sister, Amina, is a video editor who needs a computer that can process large video files quickly. Amina buys a laptop with an octa-core, high clock speed 64-bit processor, while James buys a laptop with a dual-core 64-bit processor at a lower price. Explain why their different needs led them to choose different CPUs.

REFLECT AND DISCUSS

1. How does having more CPU cores help a computer perform multiple tasks at the same time?
2. In what ways is the CPU similar to, and different from, the human brain?

Worksheet 1.5: CPU Specifications

Find the CPU specifications (brand, cores, clock speed, word length) of any three computing devices at home or school and record them in the table below.

Device	CPU Brand	Number of Cores	Clock Speed	Word Length

EXERCISE 1.5

1. Name the two main structural elements of the CPU responsible for processing data and controlling operations.
2. Explain the function of registers in a CPU.
3. Differentiate between a dual-core and a quad-core processor.
4. State three factors to consider when selecting a CPU.

END OF TOPIC QUESTIONS 1.5

1. Which part of the CPU performs arithmetic and logical operations?
A) Control Unit B) Register C) ALU D) Bus
2. What is the function of the Control Unit?
3. What does 'word length' refer to in a CPU?
4. State two factors to consider when selecting a CPU.
5. Explain the fetch-decode-execute cycle and its importance in the operation of a CPU.

(See the Answers chapter at the end of this book to check your answers.)

1.6 Operating System (OS)

Suggested duration: 12 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Describe functions of an operating system.
- Classify operating systems according to different attributes.
- Install an operating system in a computer.
- Use an operating system to perform a task.
- Acknowledge the importance of operating systems in computing.

Key Terms: Operating System, Booting, GUI, CLI, Multitasking, Multi-user

An **Operating System (OS)** is system software that manages computer hardware and software resources and provides common services for computer programs. Examples include Microsoft Windows, macOS, Linux, Android and iOS.

1.6.1 Functions of an Operating System

Function	Description
Booting	Starting up the computer and loading the OS into memory.
Resource management	Allocating hardware resources (CPU time, memory) to different programs.
User interface	Provides a way for users to interact with the computer (GUI or command line).
Memory management	Controlling how memory is allocated to and freed from programs.
I/O management	Coordinating input and output devices.
File management	Organising, storing, naming, and retrieving files.
Process management	Managing programs (processes) running on the computer, including scheduling and multitasking.
User management	Managing user accounts, permissions, and security.

1.6.2 Classification of Operating Systems

Basis	Types
-------	-------

Tasks	Single-tasking (runs one program at a time) vs Multitasking (runs several programs concurrently)
Users	Single-user (one user at a time) vs Multi-user (several users can access the system, often over a network)
User interface	Graphical User Interface (GUI - uses icons, windows, menus), Command Line Interface (CLI - uses typed text commands), Menu-driven (uses lists of options)

1.6.3 Installing and Using an Operating System

Before installing an OS, ensure the computer meets the minimum hardware requirements (processor, RAM, storage space). Installation is often carried out from a bootable USB drive or DVD, or within a virtual environment (e.g. VirtualBox or a Virtual Machine) so learners can practise safely without affecting the main computer.

Once installed, an operating system allows users to manipulate files and folders (create, rename, delete, restore, copy, move, back up), install and run application software, and manage input, output and storage devices.

IDEA BOX

Practise safely! Installing operating systems inside a **virtual machine** (a "computer within a computer" created using software like VirtualBox) lets you experiment with installing and configuring operating systems without risking damage to your real computer's files.

CASE STUDY: Choosing an Operating System for a Cyber Café

A cyber café owner needs an operating system that supports multiple users working at the same time on a network, is affordable, and is compatible with common productivity and browsing software. She settles on a multi-user, multitasking OS with a Graphical User Interface. Discuss why a single-user, single-tasking OS would not have been suitable for this business.

REFLECT AND DISCUSS

1. Which operating system do you use most often on your phone or computer? List three tasks it helps you perform.
2. Why do you think most modern operating systems use a Graphical User Interface (GUI) rather than a Command Line Interface (CLI) for everyday users?

Worksheet 1.6: File Management Practice

Using any operating system available to you, practise the following tasks and record what you did: (1) create a new folder, (2) rename it, (3) create a text file inside it, (4) copy the file to another folder, (5) delete the original file and restore it from the recycle bin/trash.

Task	Steps Followed	Outcome

EXERCISE 1.6

1. List five functions of an operating system.
2. Differentiate between a single-user and a multi-user operating system.
3. Explain the difference between a Graphical User Interface and a Command Line Interface.
4. Give three examples of operating systems used in mobile devices.
5. Explain why it is useful to install an operating system in a virtual environment before installing it on a real computer.

END OF TOPIC QUESTIONS 1.6

1. Which of the following is NOT a function of an operating system?
A) Memory management B) File management C) Designing a website
D) Process management
2. Give two examples of operating systems.
3. What is the difference between multitasking and single-tasking operating systems?
4. State one advantage of a Graphical User Interface over a Command Line Interface.
5. Discuss the importance of an operating system in the operation of a computer system.

(See the Answers chapter at the end of this book to check your answers.)

1.7 Computer Setup

Suggested duration: 10 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Explain types of ports and cables used in computers.
- Relate cables to their corresponding ports in a computer.
- Set up a computer for use.
- Appreciate following the correct procedure when setting up a computer.

Key Terms: Port, Cable, USB, HDMI, VGA, Ethernet port

1.7.1 Common Ports and Cables

Port	Matching Cable	Common Use
USB Port	USB Cable	Connecting keyboards, mice, flash drives, printers
HDMI Port	HDMI Cable	Connecting monitors, projectors, TVs (video and audio)
VGA Port	VGA Cable	Connecting older monitors/projectors (video only)
Ethernet (RJ-45) Port	Ethernet/LAN Cable	Connecting to a wired computer network
Audio Jack (3.5mm)	Audio Cable	Connecting headphones, speakers, microphones
Power Port	Power Cable	Supplying electrical power to the computer

1.7.2 Tools Needed for Computer Setup

Depending on the task, you may need a screwdriver set, anti-static wrist strap, cable ties, and a clean, static-free work surface.

1.7.3 Safety Precautions When Setting Up a Computer

- Switch off and unplug the computer before connecting or disconnecting cables inside the case.
- Handle cables and ports gently - avoid forcing a connector into the wrong port.
- Use an anti-static wrist strap to avoid damaging sensitive components with static electricity.
- Keep liquids away from the computer.
- Ensure proper ventilation to prevent overheating.
- Arrange cables neatly to avoid tripping hazards.

1.7.4 Steps in Setting Up a Computer

1. Unpack all components and identify the necessary cables and ports.
2. Connect the monitor to the system unit using the appropriate video cable (HDMI/VGA).
3. Connect the keyboard and mouse (via USB or wireless receiver).
4. Connect speakers/headphones if required.
5. Connect the network cable (if using a wired connection).
6. Connect the power cable last, then switch on the computer.
7. Confirm that the computer boots up correctly and all connected devices are recognised.

IDEA BOX

A simple way to remember safe setup order: connect everything *except power first*, then plug in power *last*, and switch on. When disconnecting, do the reverse - switch off, unplug power first, then disconnect other cables.

CASE STUDY: Setting Up a New Computer Lab

A school has just received 20 new desktop computers as a donation. The ICT club members are asked to help the technician set up the lab before the term begins. They organise themselves into pairs, each pair responsible for unpacking, connecting cables correctly, and testing one computer, while observing all safety precautions. Discuss the benefits of following a clear, step-by-step procedure like this one when setting up many computers at once.

REFLECT AND DISCUSS

1. What could happen if you connect a cable to the wrong port by force? Why is it important to always identify the correct port first?
2. Why should you always connect the power cable last when setting up a computer?

Worksheet 1.7: Ports and Cables Matching Activity

Match each port in Column A with its corresponding cable in Column B, then state one device that uses it.

Port (Column A)	Cable (Column B)	Device that uses it

EXERCISE 1.7

1. Name three types of ports found on a desktop computer.
2. State the cable used to connect a computer to a wired network.
3. List four safety precautions to observe when setting up a computer.
4. Describe, in order, the basic steps followed when setting up a desktop computer.

END OF TOPIC QUESTIONS 1.7

1. Which cable is used to connect a computer to a wired local area network?
A) HDMI cable B) VGA cable C) Ethernet cable D) USB cable
2. Why should the power cable be connected last when setting up a computer?
3. Name two devices that connect to a computer using a USB port.
4. State two safety precautions to observe when setting up a computer.
5. Explain why it is important to follow the correct procedure when setting up a computer.

(See the Answers chapter at the end of this book to check your answers.)

2.0

Computer Networking

This strand takes you into the world of computer networks - how computers communicate with one another, the media used to transmit data, the devices and elements that make up a network, and the different ways networks can be physically arranged (topologies). These concepts explain how your school's Wi-Fi, the internet, and even your phone's Bluetooth connection work.

2.1 Data Communication

Suggested duration: 6 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Define basic data communication concepts.
- Describe characteristics of data communication in computer networking.
- Analyse the components of a data communication system.
- Simulate modes of data flow in communication systems.
- Acknowledge the significance of data communication systems in networking.

Key Terms: Data communication, Protocol, Bandwidth, Simplex, Half-duplex, Full-duplex

2.1.1 Basic Concepts

Term	Meaning
Data	Raw, unprocessed facts and figures.
Signal	An electrical or electromagnetic representation of data, either analog or digital.
Communication channel	The medium through which data travels from sender to receiver.
Network	A collection of interconnected devices that can communicate and share resources.
Protocol	A set of rules that governs how data is transmitted and received between devices.
Bandwidth	The maximum amount of data that can be transmitted over a channel in a given time.

Data communication is the exchange of data between two devices via some form of transmission medium, such as a cable or wireless connection. For communication to be effective, the sender and receiver must agree on a common protocol, similar to how two people must speak the same language to understand each other.

2.1.2 Characteristics of Effective Data Communication

- **Delivery** - data must be delivered to the correct destination.
- **Accuracy** - data must be delivered without errors.
- **Timeliness** - data must be delivered within an acceptable time (especially important for audio and video).
- **Jitter** - the variation in packet arrival time should be minimised for smooth communication.

2.1.3 Components of a Data Communication System

1. **Sender** - the device that sends the data message.
2. **Message** - the data/information being communicated.
3. **Medium** - the physical or wireless path the message travels through.
4. **Protocol** - the agreed rules governing communication.
5. **Receiver** - the device that receives the message.

2.1.4 Analog and Digital Signals

Analog signals vary continuously over time (like a smooth wave), while **digital signals** represent data as discrete values, usually 0s and 1s. Modern computer networks mainly use digital signals for higher accuracy and easier processing. Key related terms: **data rate** (speed of data transfer, e.g. in bits per second) and **baud rate** (number of signal changes per second).

2.1.5 Modes of Data Flow

Mode	Description	Example
Simplex	Data flows in one direction only.	Television broadcast, keyboard to computer
Half-duplex	Data flows in both directions, but only one direction at a time.	Walkietalkie communication
Full-duplex	Data flows in both directions simultaneously.	Telephone conversation

IDEA BOX

Think of **simplex** like a one-way street, **half-duplex** like a single-lane bridge where cars take turns, and **full-duplex** like a two-lane road where traffic flows freely in both directions at once.

CASE STUDY: Video Calling Across Kenya

Wanjiru in Nairobi makes a video call to her cousin in Mombasa using a smartphone app. Both can see and hear each other at the same time without needing to take turns speaking. Identify the mode of data flow used in this video call and explain why this mode is necessary for a good video calling experience.

REFLECT AND DISCUSS

1. Give two real-life examples of simplex communication that you encounter in daily life.
2. Why do you think protocols are necessary for data communication, using an analogy from everyday life?

Worksheet 2.1: Identifying Modes of Data Flow

For each scenario, identify whether it is simplex, half-duplex, or full-duplex, and justify your answer.

Scenario	Mode of Data Flow	Justification

EXERCISE 2.1

1. Define the term 'protocol' and explain its importance in data communication.
2. Differentiate between analog and digital signals.
3. List and explain the five components of a data communication system.
4. Distinguish between half-duplex and full-duplex modes of data flow, giving an example of each.

END OF TOPIC QUESTIONS 2.1

1. Which mode of data flow allows communication in both directions at the same time?
A) Simplex B) Half-duplex C) Full-duplex D) Broadcast
2. Define the term 'bandwidth'.
3. Name the five basic components of a data communication system.
4. Give one example of a device or situation that uses simplex data flow.
5. Explain why protocols are essential in data communication, giving an everyday analogy.

(See the Answers chapter at the end of this book to check your answers.)

2.2 Data Transmission Media

Suggested duration: 8 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Define basic concepts used in data transmission.
- Describe types of transmission media used in computer networks.
- Connect digital devices used in data communication.
- Establish factors that affect communication over a computer network.
- Appreciate the role of transmission media in computer networking.

Key Terms: Transmission media, Guided media, Unguided media, Fibre optic cable, Attenuation, Multiplexing

2.2.1 Basic Concepts

Term	Meaning
Transmission	The process of sending data from one point to another.
Encoding	Converting data into a form suitable for transmission.
Decoding	Converting received signals back into their original data form.
Modulation	Superimposing a data signal onto a carrier signal for transmission.
Demodulation	Extracting the original data signal from a modulated carrier signal.
Multiplexing	Combining several signals into one for transmission over a single medium.
Demultiplexing	Separating a combined signal back into its original individual signals.

2.2.2 Types of Transmission Media

Transmission media are classified into **guided (wired)** and **unguided (wireless)** media.

Guided Media	Description
Twisted pair cable	Pairs of insulated copper wires twisted together; commonly used in LANs (e.g. UTP cable with RJ-45 connectors).
Coaxial cable	

	A central copper conductor surrounded by insulation and a metallic shield; used for cable TV and older networks.
Fibre optic cable	Uses light pulses through thin glass or plastic fibres; offers very high speed and long-distance transmission with minimal interference.

Unguided Media	Description
Radio waves	Used in Wi-Fi, radio broadcasting.
Microwaves	Used for point-to-point communication, satellite links.
Infrared	Used for short-range communication, e.g. remote controls.
Satellite	Used for long-distance communication, including television broadcasting and GPS.

2.2.3 Factors Affecting Communication over a Network (Transmission Impairment)

- **Attenuation** - the weakening of a signal as it travels over a distance.
- **Noise** - unwanted electrical signals that interfere with data signals.
- **Distortion** - a change in the shape of the signal.
- **Bandwidth limitations** - insufficient bandwidth can slow down or limit data transmission.

Ways of overcoming transmission impairments include using repeaters and amplifiers to boost signals, using shielded/quality cables, using fibre optic cables for long distances, and proper network design.

IDEA BOX

Fibre optic cables transmit data as pulses of light rather than electrical signals. Because light is not affected by electromagnetic interference, fibre optic cables can carry data over much longer distances at much higher speeds than copper cables.

CASE STUDY: Connecting a Rural School to the Internet

A rural school in Kenya is far from the nearest internet fibre cable. Instead of digging trenches for fibre optic cable, the school opts to install a satellite dish to receive internet signals wirelessly. Explain why unguided (wireless) transmission media might be more practical than guided media in this situation, and suggest one drawback of this choice.

REFLECT AND DISCUSS

1. Why do you think fibre optic cables are increasingly replacing copper cables for internet connections in Kenya?
2. Think of a situation in your community where wireless transmission media would be more practical than wired media. Explain why.

Worksheet 2.2: Guided vs Unguided Media

Classify the following as guided or unguided media, and give one advantage of each: twisted pair cable, Wi-Fi, fibre optic cable, satellite, coaxial cable, infrared.

Transmission Medium	Guided/Unguided	One Advantage

EXERCISE 2.2

1. Differentiate between guided and unguided transmission media, giving two examples of each.
2. Explain the process of multiplexing and demultiplexing.
3. Describe two factors that lead to transmission impairment.
4. Explain two ways of overcoming transmission impairment in a network.

END OF TOPIC QUESTIONS 2.2

1. Which of the following is an example of guided transmission media?
A) Radio waves B) Microwaves C) Fibre optic cable D) Satellite
2. Define the term 'attenuation'.
3. State one advantage of fibre optic cable over copper cable.
4. Explain the meaning of multiplexing.
5. Discuss the factors that would influence the choice of transmission medium for connecting a remote rural school to the internet.

(See the Answers chapter at the end of this book to check your answers.)

2.3 Computer Network Elements

Suggested duration: 10 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Identify different types of computer networks.
- Describe elements of a computer network.
- Evaluate criteria of a computer network.
- Connect a computing device to an available network.
- Appreciate the role of computer networks in communication.

Key Terms: LAN, MAN, WAN, PAN, WLAN, Router, Switch/Hub, NIC, Protocol

2.3.1 Types of Computer Networks

Network	Full Name	Coverage
PAN	Personal Area Network	Very short range, e.g. connecting a phone to wireless earphones (Bluetooth).
LAN	Local Area Network	A small area such as a single building, e.g. a school computer lab.
WLAN	Wireless Local Area Network	Similar to a LAN but connected wirelessly, e.g. Wi-Fi in a school.
MAN	Metropolitan Area Network	Covers a city or town, connecting several LANs.
WAN	Wide Area Network	Covers a large geographical area, e.g. a country or the whole world - the Internet is the largest WAN.

2.3.2 Elements of a Computer Network

- **Transmission media** - the pathway data travels through (see Topic 2.2).
- **DTE (Data Terminal Equipment)** - devices that generate or receive data, e.g. computers.
- **DCE (Data Communication Equipment)** - devices that assist in transmitting data, e.g. modems.
- **Network software** - includes network operating systems, web browsers, search engines and email clients.

2.3.3 Network Devices and Their Functions

Device	Function
--------	----------

Router	Directs data packets between different networks, e.g. connecting a home network to the internet.
Network Interface Card (NIC)	Allows a device to connect physically or wirelessly to a network.
Gateway	Connects networks that use different protocols, translating data between them.
Hub	Connects multiple devices in a network, broadcasting data to all connected devices.
Modem	Converts digital signals to analog for transmission and back again (Modulator-Demodulator).
Bridge	Connects and filters traffic between two network segments.
Repeater	Boosts/regenerates a weakened signal so it can travel further.

2.3.4 Network Protocols

Protocols are standards that govern communication over a network, e.g. **TCP/IP** (basic protocol suite of the internet), **HTTP/HTTPS** (for web pages), **SMTP** (for sending email), **FTP** (for transferring files).

2.3.5 Qualities of a Good Computer Network

- **Performance** - how fast and efficiently data is transmitted.
- **Security** - protection against unauthorised access.
- **Scalability** - the ability to grow and accommodate more devices/users.
- **Reliability** - consistent, error-free operation.
- **Availability** - the network is accessible when needed.

2.3.6 Methods of Connecting to a Network

Point-to-point connection links exactly two devices directly, while **multi-point connection** links more than two devices sharing the same communication channel.

IDEA BOX

Remember the size order from smallest to largest: **PAN - LAN/WLAN - MAN - WAN**. The Internet is the best-known example of a WAN, connecting millions of smaller networks worldwide.

CASE STUDY: Networking a Cyber Café

A cyber café owner connects 10 computers to a single router, which is in turn connected to the internet via a modem. All computers can access the internet and can print to a shared printer connected through the network. Identify the type of network formed within the cyber café and explain the role played by the router and modem in this setup.

REFLECT AND DISCUSS

1. Identify examples of PAN, LAN, and WAN that you interact with in your everyday life.
2. Why is network security an important quality for any organisation's network, such as a bank or a school?

Worksheet 2.3: Network Devices and Their Functions

Match each network device to its correct function and identify one real-life location where you might find it.

Device	Function	Where You Might Find It

EXERCISE 2.3

1. Differentiate between a LAN and a WAN, giving one example of each.
2. State the functions of a router and a modem in a computer network.
3. List the five qualities of a good computer network.
4. Differentiate between a point-to-point and a multi-point connection.

END OF TOPIC QUESTIONS 2.3

1. Which type of network covers the largest geographical area?
A) PAN B) LAN C) MAN D) WAN
2. State the function of a router in a computer network.
3. Give an example of a network protocol used for sending email.
4. List any three qualities of a good computer network.
5. Explain the role of network elements (transmission media, DTE, DCE, network software) in forming a functional computer network, using an example of a cyber café.

(See the Answers chapter at the end of this book to check your answers.)

2.4 Network Topologies

Suggested duration: 8 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Differentiate between physical and logical network topologies.
- Describe types of logical and physical topologies in computer networking.
- Create a physical network topology for a computer network.
- Appreciate the use of network topologies in computer networks.

Key Terms: Topology, Star topology, Bus topology, Ring topology, Mesh topology, Tree topology, Hybrid topology

2.4.1 Physical vs Logical Topology

A **physical topology** describes the actual physical layout of cables and devices in a network. A **logical topology** describes how data actually flows within the network, regardless of its physical layout.

2.4.2 Types of Physical Topologies

Topology	Description	Advantages	Disadvantages
Bus	All devices connect to a single central cable (the "bus").	Simple, cheap to install.	If the main cable fails, the whole network fails; slows down as more devices are added.
Star	All devices connect to a central device (hub or switch).	Easy to install and manage; failure of one cable/device does not affect others.	If the central device fails, the whole network fails.
Ring	Devices are connected in a closed loop, data travels in one direction around the ring.	Data flows efficiently in one direction; equal access for all devices.	Failure of one device/cable can disrupt the entire network.
Mesh	Every device is connected to every other device.	Highly reliable - multiple paths mean no single point of failure.	Expensive and complex to install due to the large number of connections.
Tree	Combines multiple star networks connected to a central bus (hierarchical structure).	Easy to expand; good for organising large networks.	If the backbone cable fails, connected segments are affected.
Hybrid			

	A combination of two or more different topologies.	Flexible and scalable, combines advantages of different topologies.	Can be complex and costly to design and manage.
--	--	---	---

2.4.3 Choosing an Appropriate Topology

The choice of topology depends on: the size of the network, cost, required reliability, ease of installation and maintenance, and the physical layout of the building.

IDEA BOX

Most modern school and office LANs use a **star topology** because a single cable or device failure does not bring down the whole network - only the one affected device.

CASE STUDY: Designing a Network for a New School Block

A school is setting up a computer lab with 30 computers in one room, connected to a central switch. The switch, in turn, connects to switches in two other blocks, forming a hierarchical structure connected to the school's main server room. Identify the physical topology used within the computer lab, and the overall topology used to connect the different blocks of the school. Explain one advantage of this design for the school.

REFLECT AND DISCUSS

1. If you were designing a network for your own school, which topology would you choose and why?
2. Explain the difference between a physical topology and a logical topology using an example.

Worksheet 2.4: Drawing Network Topologies

Draw a labelled diagram of each of the following topologies: bus, star, ring, and mesh. Under each diagram, write one advantage and one disadvantage.

Topology	Advantage	Disadvantage

EXERCISE 2.4

1. Differentiate between a physical topology and a logical topology.
2. Describe the star topology and state one advantage and one disadvantage.
3. Explain why a mesh topology, though reliable, is rarely used in small networks such as school labs.
4. Suggest a suitable topology for connecting computers in a single small office of five computers, giving a reason.

END OF TOPIC QUESTIONS 2.4

1. In which topology does a single central device connect to all other devices?
A) Bus B) Ring C) Star D) Mesh
2. State one disadvantage of the bus topology.
3. What is a hybrid topology?
4. Give one advantage of the mesh topology.
5. Discuss the factors a school should consider when choosing a network topology for its computer laboratory.

(See the Answers chapter at the end of this book to check your answers.)

3.0

Software Development

This strand introduces you to computer programming - the process of writing instructions that tell a computer what to do. You will learn about programming languages, how to design solutions using algorithms, and how to write actual programs using variables, operators, control structures, data structures, and functions.

3.1 Computer Programming Concepts

Suggested duration: 14 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Explain the terminologies used in programming languages.
- Describe the evolution of programming languages in software development.
- Categorise programming languages according to the paradigms.
- Create simple instructions to simulate low level programming languages.
- Acknowledge the evolution of programming languages in software development.

Key Terms: Programming language, Compiler, Interpreter, Assembler, Syntax, Paradigm

3.1.1 Basic Terminologies

Term	Meaning
Programming	The process of writing a set of instructions that direct a computer to perform a specific task.
Programming language	A formal language with rules used to write computer programs, e.g. Python, Java, C++.
Source code	The instructions written by a programmer in a programming language, in humanreadable form.
Machine code	Instructions written in binary (0s and 1s) that the CPU can execute directly.
Assembler	A program that translates assembly language (using mnemonics) into machine code.
Compiler	A program that translates the entire source code of a high-level language into machine code before execution.
Interpreter	A program that translates and executes source code line by line, rather than all at once.
Syntax	The set of rules that define the correct structure of statements in a programming language.
Integrated Development Environment (IDE)	Software that provides tools (editor, compiler/ interpreter, debugger) for writing and testing programs, e.g. PyCharm, Visual Studio Code.
Software Development Life Cycle (SDLC)	The structured process followed to plan, create, test and deploy software.

3.1.2 Evolution and Levels of Programming Languages

Level	Description	Example
Machine language (1st generation)	Written directly in binary; understood directly by the CPU but very difficult for humans to write and read.	10110000 01100001
Assembly language (2nd generation)	Uses short, human-readable mnemonics (e.g. ADD, MOV, SUB) that correspond directly to machine instructions; requires an assembler to translate to machine code.	MOV AX, 5
High-level languages (3rd generation onwards)	Use English-like syntax that is easier for humans to write; requires a compiler or interpreter to translate to machine code.	Python, Java, C++, C#

3.1.3 Programming Paradigms

Paradigm	Description
Structured/Procedural	Programs are organised as a sequence of instructions/procedures executed in order, e.g. C.
Object-Oriented	Programs are organised around "objects" that combine data and behaviour, e.g. Java, Python, C++.
Functional	Programs are built by composing functions and avoiding changing state, e.g. Haskell.
Logic	Programs express facts and rules, and the computer derives conclusions, e.g. Prolog.
Event-driven	Program flow is determined by events such as user actions (clicks, key presses), e.g. JavaScript, visual programming environments like Scratch.

IDEA BOX

A useful way to remember the difference: A **compiler** is like translating an entire book from English to Kiswahili before anyone reads it, while an **interpreter** is like a live translator who translates sentence by sentence as the speaker talks.

CASE STUDY: From Machine Code to Python

In the 1950s, programmers had to write instructions in pure machine code (binary) or assembly language, which took a long time and was very error-prone. Today, a Grade 10 learner can write a simple program to add two numbers in Python in just a few lines of easy-to-read code. Discuss how the evolution of programming languages from low-level to high-level has made software development more accessible to more people.

REFLECT AND DISCUSS

1. Why do you think most modern software is written in high-level languages rather than assembly or machine language?
2. Which programming paradigm do you think would be most suitable for creating a simple mobile game, and why?

Worksheet 3.1: Programming Languages Timeline

Research and create a timeline chart showing at least 6 programming languages in the order they were developed, noting the paradigm each one is best known for.

Programming Language	Year Developed (approx.)	Paradigm

EXERCISE 3.1

1. Differentiate between a compiler and an interpreter.
2. Explain the difference between low-level and high-level programming languages.
3. Describe any three programming paradigms.
4. Explain the role of an Integrated Development Environment (IDE) in programming.

END OF TOPIC QUESTIONS 3.1

1. Which type of language translator translates source code line by line during execution?
A) Assembler B) Compiler C) Interpreter D) Editor
2. State one example of a low-level programming language.
3. Define the term 'syntax' as used in programming.
4. Name any two programming paradigms.
5. Explain the evolution of programming languages from machine language to high-level languages, and its impact on software development.

(See the Answers chapter at the end of this book to check your answers.)

3.2 Program Development

Suggested duration: 15 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Describe stages of program development in computer programming.
- Write a pseudocode to illustrate the logical flow of an algorithm.
- Represent the logical flow of an algorithm using a flowchart.
- Design an algorithm to solve a real life problem.
- Appreciate the importance of using algorithms in problem solving.

Key Terms: Algorithm, Pseudocode, Flowchart, Program development cycle

3.2.1 Stages of Program Development

1. **Problem definition** - clearly understanding what the program should do.
2. **Program design** - planning the solution using algorithms, pseudocode and/or flowcharts.
3. **Coding** - writing the actual program using a programming language.
4. **Testing** - running the program with sample data to check it works correctly and fixing errors (debugging).
5. **Implementation** - putting the finished program into actual use.
6. **Documentation** - writing clear notes explaining how the program works, for future reference.
7. **Maintenance** - updating the program over time to fix issues or add new features.

3.2.2 Algorithms

An **algorithm** is a finite, step-by-step set of instructions for solving a problem. A good algorithm has the following characteristics:

- **Input** - it should accept zero or more inputs.
- **Output** - it should produce at least one output.
- **Finiteness** - it must terminate after a finite number of steps.
- **Definiteness** - each step must be clearly and unambiguously defined.
- **Effectiveness** - each step must be simple enough to be carried out.

3.2.3 Pseudocode

Pseudocode is a way of writing an algorithm using plain, structured English-like statements, without following the strict syntax of any particular programming language. Common keywords include: **START** , **INPUT** , **PROCESS** (or a calculation), **OUTPUT** , **END** .

Example: Pseudocode to find the sum of two numbers:

```
START
  INPUT num1, num2
  sum = num1 + num2
  OUTPUT sum
END
```

3.2.4 Flowcharts

A flowchart uses standard symbols to represent the logical flow of an algorithm visually.

Symbol Name	Shape	Purpose
Terminator	Oval	Marks the Start/End of a flowchart.
Input/Output	Parallelogram	Represents input or output of data.
Process	Rectangle	Represents a processing step or calculation.
Decision	Diamond	Represents a decision point with a Yes/No or True/False branch.
Flow line/Arrow	Arrow	Shows the direction/sequence of steps.
Connector	Circle	Connects different parts of a flowchart, especially across pages.

IDEA BOX

Always start designing a solution to a problem with pseudocode or a flowchart **before** writing actual program code. This helps you plan your logic clearly and catch errors early, saving time later!

CASE STUDY: Automating the Tuck Shop Till

A school tuck shop wants a simple program that calculates the total cost of items bought by a student and the balance to give back after payment. Before writing any code, the ICT club first draws a flowchart showing: Start - Input price of items and amount paid - Calculate total cost - Calculate balance - Output balance - End. Explain why planning the algorithm first, using a flowchart, is a useful step before the actual coding begins.

REFLECT AND DISCUSS

1. Why is testing considered such an important stage in program development, even after the program appears to work?
2. Think of a daily task you do (e.g. making tea, getting ready for school). Break it down into a simple step-by-step algorithm.

Worksheet 3.2: Designing Algorithms

For the problem 'Find the average of three numbers entered by a user', write (a) a pseudocode, and (b) draw a flowchart representing the solution.

EXERCISE 3.2

1. List, in order, the seven stages of the program development cycle.
2. State the five characteristics of a good algorithm.
3. Write a pseudocode to determine whether a number entered by a user is even or odd.
4. Draw a flowchart to represent an algorithm that finds the largest of two numbers.

END OF TOPIC QUESTIONS 3.2

1. Which flowchart symbol is used to represent a decision point?
A) Oval B) Rectangle C) Diamond D) Parallelogram
2. Define the term 'algorithm'.
3. List the stages of the program development cycle in the correct order.
4. State two characteristics of a good algorithm.
5. Write a pseudocode and describe a flowchart for an algorithm that accepts a student's marks and determines whether the student has passed (marks ≥ 50) or failed.

(See the Answers chapter at the end of this book to check your answers.)

3.3 Identifiers and Operators

Suggested duration: 18 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Describe the elementary elements of a computer program.
- Declare variables and constants in a programming language.
- Use input and output statements in a programming language.
- Use operators in a programming language.
- Appreciate the role of identifiers and operators in programming.

Key Terms: Variable, Constant, Data type, Identifier, Reserved word, Operator

3.3.1 Elementary Elements of a Program

Term	Meaning
Reserved word (keyword)	A word that has special meaning in a programming language and cannot be used as an identifier, e.g. <code>if</code> , <code>while</code> , <code>print</code> in Python.
Identifier	A name given by the programmer to a variable, constant, function, or other program element, e.g. <code>totalMarks</code> .
Data type	Classifies the kind of value a variable can hold, e.g. integer, float, string, boolean.
Syntax error	An error that occurs when the rules of the programming language are not followed correctly.

3.3.2 Variables and Constants

A **variable** is a named storage location whose value can change during program execution. A **constant** is a named storage location whose value does not change once set.

Example in Python:

```
age = 15           # variable - can change
PI = 3.14159      # constant (by convention, written in uppercase)
name = "Amina"    # variable holding a string
```

3.3.3 Common Data Types

Data Type	Description	Example
-----------	-------------	---------

Integer (int)	Whole numbers	10, -5, 200
Float	Decimal (real) numbers	3.14, -0.5
String (str)	Text/characters	"Computer Science"
Boolean (bool)	True/False values	True, False

3.3.4 Input and Output Statements

Input statements collect data from the user; output statements display information. In Python:

```
name = input("Enter your name: ") # input statement
print("Hello,", name)           # output statement
```

3.3.5 Operators

Category	Examples	Purpose
Arithmetic	+, -, *, /, % (modulus), ** (exponent)	Performing mathematical calculations.
Assignment	=, +=, -=, *=, /=	Assigning or updating values in variables.
Increment/Decrement	++, -- (or i += 1 in Python)	Increasing or decreasing a value by one.
Relational (Comparison)	==, !=, >, <, >=, <=	Comparing two values, resulting in True or False.
Logical	AND, OR, NOT	Combining or reversing boolean (True/False) conditions.

3.3.6 Order of Precedence

Just like in mathematics, operators follow an order of precedence: parentheses first, then exponents, then multiplication/division, then addition/subtraction, and finally relational and logical operators. For example, in `2 + 3 * 4`, multiplication happens first, giving `2 + 12 = 14`.

IDEA BOX

A common mistake for beginners is confusing the assignment operator `=` (which stores a value) with the relational operator `==` (which compares two values). Remember: one equal sign assigns, two equal signs compare!

CASE STUDY: A Simple Grading Program

A learner writes a Python program that asks the user to input their marks (an integer variable), then uses a relational operator to compare the marks to 50 to decide whether they passed or failed. Identify the variable, data type, and relational operator used in this scenario, and explain their roles.

REFLECT AND DISCUSS

1. Why do you think programming languages restrict what characters can be used in an identifier (variable name)?
2. Explain, using an example, the difference between a variable and a constant.

Worksheet 3.3: Identifying Data Types and Operators

For each of the following values, state its most likely data type, then write one program statement that uses an arithmetic operator involving that value: 25, 'Nairobi', 3.75, True.

Value	Data Type	Sample Statement

EXERCISE 3.3

1. Differentiate between a variable and a constant, giving an example of each.
2. List the four basic data types commonly used in programming.
3. Write a Python statement that inputs a user's age and stores it in a variable called 'age'.
4. Evaluate the expression: $10 + 2 * 5 - 3$, showing the order of operations.
5. Differentiate between relational and logical operators, giving an example of each.

END OF TOPIC QUESTIONS 3.3

1. Which of the following is a valid identifier in most programming languages?
A) 2total B) total marks C) total_marks D) total-marks
2. What is the data type of the value 'True'?
3. Write the Python assignment operator used to add a value to a variable and store the result back in the same variable.
4. State the result of the expression `15 % 4`.
5. Explain the difference between the assignment operator (`=`) and the equality/relational operator (`==`), giving an example of each.

(See the Answers chapter at the end of this book to check your answers.)

3.4 Control Structures

Suggested duration: 17 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Describe control structures in programming.
- Select program control structure for a given situation.
- Use control structures in programming.
- Appreciate the application of control structures in programming.

Key Terms: Sequential structure, Selection structure, Iteration structure, Boolean expression, Loop

3.4.1 What are Control Structures?

Control structures determine the order in which statements in a program are executed. There are three main types:

Structure	Description	Example Keywords (Python)
Sequential	Statements are executed one after another, in the order they appear.	(default flow)
Selection (Decision)	The program chooses between different paths based on a condition.	<code>if</code> , <code>elif</code> , <code>else</code>
Iteration (Repetition/ Loop)	A block of statements is repeated multiple times.	<code>for</code> , <code>while</code>

3.4.2 Selection Structures

Selection structures use **Boolean expressions** (expressions that evaluate to True or False) to decide which path to follow.

```
marks = int(input("Enter your marks: "))
if marks >= 50:
    print("You passed")
else:
    print("You failed")
```

Multiple conditions can be checked using `elif` (else if):

```
if marks >= 80:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

3.4.3 Iteration (Loop) Structures

A **for loop** repeats a block of code a known number of times:

```
for i in range(1, 6):
    print("Number:", i)
```

A **while loop** repeats a block of code as long as a condition remains True:

```
count = 1
while count <= 5:
    print("Count is", count)
    count += 1
```

3.4.4 Break and Continue Statements

The **break** statement immediately exits a loop. The **continue** statement skips the rest of the current loop iteration and moves to the next one.

```
for i in range(1, 10):
    if i == 5:
        break          # stops the loop when i equals 5
    print(i)
```

3.4.5 Compound (Logical) Expressions

Multiple conditions can be combined using logical operators: **and** (both must be True), **or** (at least one must be True), **not** (reverses a condition).

```
if age >= 13 and age <= 19:
    print("You are a teenager")
```

IDEA BOX

Choose a **for loop** when you know exactly how many times you want to repeat an action (e.g. "print numbers 1 to 10"). Choose a **while loop** when repetition depends on a condition that may not have a fixed number of repeats (e.g. "keep asking for a password until the correct one is entered").

CASE STUDY: An ATM PIN Verification System

An ATM machine keeps asking a customer to re-enter their PIN if it is incorrect, but locks the card after three failed attempts. Identify which control structures (selection and/or iteration) would be needed to implement this behaviour, and explain how they would work together.

REFLECT AND DISCUSS

1. Think of a real-life situation that works like a 'while loop' - something that keeps repeating until a condition changes.
2. Why is it useful to be able to combine two conditions using 'and' or 'or' in a program?

Worksheet 3.4: Choosing the Right Control Structure

For each described scenario, state whether a sequential, selection, or iteration structure would be most appropriate, and justify your answer.

Scenario	Control Structure	Justification

EXERCISE 3.4

1. Name and briefly describe the three main types of control structures.
2. Differentiate between a 'for' loop and a 'while' loop, giving an example of when each would be used.
3. Explain the difference between the 'break' and 'continue' statements.
4. Write a short pseudocode using a selection structure to determine if a number is positive, negative, or zero.

END OF TOPIC QUESTIONS 3.4

1. Which control structure would you use to repeat an action exactly 10 times?
A) Sequential B) Selection C) for loop D) if-else
2. State the purpose of the 'break' statement in a loop.
3. Write the Python keyword used to check an alternative condition after an initial 'if' statement.
4. Give an example of a real-life situation that could be represented using a selection structure.
5. Using an ATM PIN verification example, explain how selection and iteration structures can be combined to solve a real-world problem.

(See the Answers chapter at the end of this book to check your answers.)

3.5 Data Structures (Containers)

Suggested duration: 14 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Describe types of containers in programming.
- Use containers in programming.
- Apply sorting and searching techniques in data structures.
- Embrace the use of containers in programming.

Key Terms: List/Array, Dictionary, Set, Tuple, Sequential search, Binary search, Bubble sort

3.5.1 What are Containers (Data Structures)?

A container (data structure) is used to store and organise a collection of related values under a single identifier, rather than using many separate variables.

Container	Description	Example (Python)
List/Array	An ordered, changeable collection of items, accessed by position (index).	<code>fruits = ["mango", "banana", "apple"]</code>
Tuple	An ordered collection of items that cannot be changed after creation.	<code>coordinates = (2, 5)</code>
Dictionary	A collection of key-value pairs, where each key maps to a specific value.	<code>student = {"name": "Amina", "age": 15}</code>
Set	An unordered collection of unique items (no duplicates allowed).	<code>colours = {"red", "green", "blue"}</code>

An **array** can be **one-dimensional** (a simple list of values) or **two-dimensional** (a table-like structure of rows and columns, e.g. representing a chess board or a spreadsheet).

3.5.2 Accessing Elements

Elements in a list/array are accessed using an **index**, which usually starts at 0.

```
fruits = ["mango", "banana", "apple"]
print(fruits[0])    # displays "mango"
print(fruits[2])    # displays "apple"
```

3.5.3 Searching Techniques

Technique	Description
Sequential (Linear) Search	Checks each element in a list, one by one, from the beginning until the target value is found or the list ends. Works on both sorted and unsorted lists but can be slow for large lists.
Binary Search	Repeatedly divides a sorted list in half, comparing the target value to the middle element, and eliminating the half that cannot contain the target. Much faster than sequential search for large sorted lists.

3.5.4 Sorting Techniques

Technique	Description
Bubble Sort	Repeatedly compares adjacent elements and swaps them if they are in the wrong order, causing larger values to "bubble" to the end of the list.
Selection Sort	Repeatedly selects the smallest (or largest) remaining element and moves it to its correct position.
Insertion Sort	Builds a sorted list one item at a time, inserting each new element into its correct position among those already sorted.
Quick Sort	Selects a "pivot" element and partitions the list into elements smaller and larger than the pivot, then recursively sorts each partition. Generally very fast for large lists.

IDEA BOX

Binary search only works on a **sorted** list! Always sort your data first (e.g. using bubble sort, selection sort, or another method) before applying binary search.

CASE STUDY: Organising Student Records

A teacher keeps a list of 40 students' names and marks. To find a particular student's mark quickly, the teacher first sorts the list of names alphabetically, then uses binary search to locate any given student's name much faster than checking each name one by one. Explain why sorting the list first made searching more efficient in this case.

REFLECT AND DISCUSS

1. Why might a dictionary (key-value pairs) be more useful than a simple list for storing information about a student (e.g. name, age, class)?
2. Explain, in your own words, why binary search is generally faster than sequential search on a large sorted list.

Worksheet 3.5: Working with Containers

Given the list of marks [45, 78, 62, 90, 55, 33], (a) sort the list in ascending order showing your steps using bubble sort logic, and (b) use sequential search to find the position of 62.

EXERCISE 3.5

1. Differentiate between a list and a tuple in programming.
2. Describe the structure of a dictionary, giving an example.
3. Explain the difference between sequential search and binary search.
4. Describe how bubble sort arranges a list of numbers in ascending order.

END OF TOPIC QUESTIONS 3.5

1. Which container stores data as unique, unordered items with no duplicates?
A) List B) Tuple C) Set D) Dictionary
2. What is the index of the first element in a list in most programming languages?
3. State one condition that must be met before binary search can be used.
4. Name any two sorting techniques.
5. Explain, with reference to a real-life example, why data structures (containers) are important in programming.

(See the Answers chapter at the end of this book to check your answers.)

3.6 Functions

Suggested duration: 14 lessons

LEARNING OUTCOMES: By the end of this topic, you should be able to:

- Discuss types of functions used in modular programming.
- Use built-in and user-defined functions to create a modular program.
- Discuss the scope of variables and parameter passing between functions.
- Appreciate the importance of modularity in programming.

Key Terms: Function, Parameter, Argument, Return value, Local variable, Global variable, Modular programming

3.6.1 Modular Programming and Functions

Modular programming is the practice of breaking a large program into smaller, manageable, and reusable blocks called **functions** (or modules). This makes programs easier to write, test, debug, and maintain.

3.6.2 Types of Functions

Type	Description	Example
Built-in (library) function	Provided by the programming language itself; ready to use.	<code>print()</code> , <code>len()</code> , <code>input()</code> in Python
User-defined function	Created by the programmer to perform a specific task needed by their program.	<code>def calculate_total():</code>

3.6.3 Structure of a Function

A function generally has: a **return type** (the type of value it gives back, if any), a **function name**, a **parameter list** (inputs the function expects), and a **function body** (the statements that perform the task).

```
def add_numbers(a, b):           # function name: add_numbers, parameters: a, b
    total = a + b
    return total                 # return statement sends back the result

result = add_numbers(4, 7)      # calling the function with arguments 4 and 7
print(result)                   # displays 11
```


3.6.4 Parameter Passing

Term	Meaning
Formal parameter	The variable listed in the function's definition (e.g. <code>a</code> , <code>b</code> above).
Actual parameter (argument)	The actual value supplied when the function is called (e.g. <code>4</code> , <code>7</code> above).
Function prototype/ signature	A declaration showing a function's name, parameters, and return type, without the full body.

3.6.5 Scope of Variables

Scope	Description
Local variable	Declared inside a function; can only be accessed within that function.
Global variable	Declared outside all functions; can be accessed from anywhere in the program.

IDEA BOX

Think of a function like a small kitchen appliance, e.g. a blender: you give it ingredients (parameters), it processes them internally, and gives you back a smoothie (the return value) - you don't need to know exactly what happens inside to use it!

CASE STUDY: Building a School Fees Calculator

A programmer building a school fees management system creates separate functions: one to calculate tuition fees, one to calculate transport fees, and one to calculate the total amount payable by combining the results of the first two functions. Explain how breaking the program into these three functions makes the system easier to test, maintain, and reuse compared to writing all the logic in one long block of code.

REFLECT AND DISCUSS

1. Why is it good practice to break a large program into smaller functions rather than writing everything in one big block of code?
2. Can you think of a real-world 'function' (something that takes an input and produces an output) outside of programming?

Worksheet 3.6: Designing Functions

Design a user-defined function called 'calculate_area' that accepts the length and width of a rectangle as parameters and returns its area. Write the pseudocode, then identify the formal parameters and an example set of actual parameters (arguments).

EXERCISE 3.6

1. Differentiate between a built-in function and a user-defined function, giving one example of each.
2. Explain the meaning of 'formal parameter' and 'actual parameter', using an example.
3. Differentiate between a local variable and a global variable.
4. Explain the importance of modular programming, giving two advantages.

END OF TOPIC QUESTIONS 3.6

1. Which of the following is an example of a built-in function in Python?
A) calculate_total() B) print() C) add_numbers() D) myFunction()
2. What is a local variable?
3. Define the term 'parameter' as used in functions.
4. State one advantage of using functions (modular programming) in software development.
5. Using an example, explain how user-defined functions support modular programming and improve software development.

(See the Answers chapter at the end of this book to check your answers.)

Answers to End of Topic Questions

Use this chapter to check your work after attempting each End of Topic Questions section.

STRAND 1.0: Foundation of Computer Science

1.1 Evolution and Development of Computers

1. **Q1.** B
2. **Q2.** Charles Babbage
3. **Q3.** Integrated Circuits (ICs)
4. **Q4.** Any two of: large in size, used vacuum tubes, consumed a lot of power, generated a lot of heat, were slow and expensive.
5. **Q5.** VLSI technology made it possible to fit hundreds of thousands to millions of transistors onto a single chip. This miniaturisation led directly to the creation of the microprocessor, a single chip capable of performing all the functions of a computer's central processing unit, which made personal computers possible.

1.2 Computer Organisation and Architecture

1. **Q1.** stored-program
2. **Q2.** C
3. **Q3.** Fetch, Decode, Execute
4. **Q4.** 11 ($8+0+2+1 = 11$)
5. **Q5.** RISC uses a small set of simple instructions that generally execute in one clock cycle, while CISC uses a larger set of complex instructions that may take several clock cycles. RISC processors (e.g. ARM) are commonly used in mobile devices for energy efficiency, while CISC processors (e.g. Intel/AMD) are common in desktop computers for handling complex tasks.

1.3 Input/Output (I/O) Devices

1. **Q1.** C
2. **Q2.** A QR code can store much more data (text, links) and can be scanned using an ordinary smartphone camera without special equipment.
3. **Q3.** Touch screen
4. **Q4.** Any four of: purpose/task, speed, cost, compatibility, ease of use, accuracy.
5. **Q5.** Input devices such as barcode/QR scanners allow data about books and borrowers to be captured quickly and accurately compared to manual writing, reducing errors and time taken. Output devices such as monitors and printers allow librarians to instantly view records and print reports, making the borrowing and returning process faster and more reliable, as illustrated in the school library case study.

1.4 Computer Storage

1. **Q1.** B
2. **Q2.** Hard Disk Drive (HDD) or Solid State Drive (SSD)
3. **Q3.** SSDs have faster read/write speeds and are more durable since they have no moving parts.
4. **Q4.** Remote storage refers to storing data on servers accessed over the internet rather than on a local device, e.g. Google Drive, Dropbox, OneDrive.
5. **Q5.** Capacity - the device must hold enough data for all student records; Data security - the device or service should protect sensitive information from unauthorised access, e.g. through passwords/encryption; Durability/robustness - the device should be reliable and not easily damaged, reducing the risk of data loss. Portability and cost may also be considered depending on the school's needs.

1.5 Central Processing Unit (CPU)

1. **Q1.** C
2. **Q2.** It directs and coordinates the activities of the computer, fetching, decoding and managing the execution of instructions.
3. **Q3.** The amount of data the CPU can process in one cycle, e.g. 16-bit, 32-bit or 64-bit.
4. **Q4.** Any two of: clock speed, number of cores, cache size, compatibility, power consumption, cost.
5. **Q5.** The fetch-decode-execute cycle is the process by which the CPU retrieves an instruction from memory (fetch), interprets what the instruction means (decode), and carries it out (execute). This cycle repeats continuously and is important because it is the basic mechanism through which the CPU processes every instruction in a program, enabling the computer to perform tasks.

1.6 Operating System (OS)

1. **Q1.** C
2. **Q2.** Any two of: Windows, macOS, Linux, Android, iOS, Chrome OS
3. **Q3.** A multitasking OS can run several programs at the same time, while a single-tasking OS can run only one program at a time.
4. **Q4.** A GUI is easier and more user-friendly for beginners because it uses visual elements (icons, windows, menus) instead of requiring users to memorise and type text commands.
5. **Q5.** The operating system is essential because it manages all hardware and software resources (memory, processing, storage, input/output devices), provides a user interface for interacting with the computer, manages files, and coordinates processes/users. Without an OS, application programs would have no way to communicate with the computer's hardware, so the computer could not function usefully.

1.7 Computer Setup

1. **Q1.** C
2. **Q2.** To reduce the risk of electric shock or damage to components while other cables are still being connected.

3. **Q3.** Any two of: keyboard, mouse, flash drive, printer.
4. **Q4.** Any two of: switch off and unplug before connecting internal cables, use an anti-static wrist strap, keep liquids away, ensure proper ventilation, arrange cables to avoid tripping.
5. **Q5.** Following the correct procedure ensures that all components are connected properly so the computer functions correctly, reduces the risk of damaging sensitive hardware through incorrect connections or static electricity, and protects the safety of the person setting up the computer, e.g. by avoiding electric shock.

STRAND 2.0: Computer Networking

2.1 Data Communication

1. **Q1.** C
2. **Q2.** The maximum amount of data that can be transmitted over a communication channel in a given period of time.
3. **Q3.** Sender, message, medium, protocol, receiver.
4. **Q4.** Television broadcasting, or data flow from a keyboard to a computer.
5. **Q5.** Protocols are sets of rules that ensure devices on a network understand each other and exchange data correctly and reliably, similar to how two people need to speak the same language and follow conversational norms (e.g. taking turns) to communicate effectively. Without agreed protocols, devices from different manufacturers or using different technologies would not be able to exchange data successfully.

2.2 Data Transmission Media

1. **Q1.** C
2. **Q2.** The weakening of a signal's strength as it travels over a distance through a transmission medium.
3. **Q3.** Fibre optic cable transmits data at much higher speeds, over longer distances, and is not affected by electromagnetic interference.
4. **Q4.** The process of combining several data signals into one signal for transmission over a single communication channel/medium.
5. **Q5.** Factors include: cost (wireless/satellite may avoid the expense of laying cables over long distances); terrain (guided media may be difficult to lay in hilly or remote areas); required speed and bandwidth; distance from existing infrastructure; and reliability. In many remote areas, unguided media such as satellite or microwave links are more practical since they do not require physical cables to be laid across long, difficult terrain.

2.3 Computer Network Elements

1. **Q1.** D
2. **Q2.** It directs data packets between different networks, e.g. connecting a local network to the internet.
3. **Q3.** SMTP
4. **Q4.** Any three of: performance, security, scalability, reliability, availability.

5. **Q5.** Transmission media (e.g. cables or Wi-Fi) provide the pathway for data to travel; DTE such as the customers' computers generate and receive data; DCE such as the modem and router assist in transmitting data between the local network and the internet; and network software (network OS, browsers) allows users to access and use network services. In a cyber café, all these elements work together so that customers' computers can connect to a shared router/modem and access the internet and shared resources like a printer.

2.4 Network Topologies

1. **Q1.** C
2. **Q2.** If the main cable fails, the entire network is affected/fails.
3. **Q3.** A network topology that combines two or more different types of topologies.
4. **Q4.** It is highly reliable since there are multiple paths for data, so failure of one connection does not disrupt the network.
5. **Q5.** The school should consider: cost of cabling and devices, the number of computers to be connected, reliability needed (whether failure of one device should affect others), ease of installation and future expansion, and the physical layout of the lab. For example, a star topology is often preferred in school labs because it is easy to install, manage, and expand, and a fault in one computer's cable does not bring down the whole network.

STRAND 3.0: Software Development

3.1 Computer Programming Concepts

1. **Q1.** C
2. **Q2.** Machine language or Assembly language
3. **Q3.** The set of rules that define the correct structure/grammar of statements in a programming language.
4. **Q4.** Any two of: structured/procedural, object-oriented, functional, logic, event-driven.
5. **Q5.** Programming began with machine language (pure binary), which was difficult and error-prone. Assembly language introduced mnemonics that were easier for humans but still closely tied to hardware, requiring an assembler. High-level languages then introduced English-like syntax, making programs easier to write, read, and maintain, and portable across different computers using compilers or interpreters. This evolution has made software development faster, more accessible to more people, and has enabled the creation of complex modern software.

3.2 Program Development

1. **Q1.** C
2. **Q2.** A finite, step-by-step set of instructions for solving a problem.
3. **Q3.** Problem definition, program design, coding, testing, implementation, documentation, maintenance.
4. **Q4.** Any two of: has input, has output, finiteness, definiteness, effectiveness.

5. **Q5.** Pseudocode: START; INPUT marks; IF marks $\geq$ 50 THEN OUTPUT 'Pass' ELSE OUTPUT 'Fail'; END. The flowchart would use: an oval for Start, a parallelogram to input marks, a diamond to test the condition marks $\geq$ 50, with the Yes path leading to a parallelogram outputting 'Pass' and the No path leading to a parallelogram outputting 'Fail', both leading to an oval marked End.

3.3 Identifiers and Operators

1. **Q1.** C
2. **Q2.** Boolean
3. **Q3.** +=
4. **Q4.** 3 (15 divided by 4 leaves a remainder of 3)
5. **Q5.** The assignment operator (=) is used to store a value into a variable, e.g. age = 15 stores the value 15 in the variable age. The equality/relational operator (==) is used to compare two values and returns True or False, e.g. age == 15 checks whether the value stored in age is equal to 15. Confusing the two is a common programming error.

3.4 Control Structures

1. **Q1.** C
2. **Q2.** It immediately stops/exits the loop, regardless of the loop's condition.
3. **Q3.** elif
4. **Q4.** Any reasonable example, e.g. deciding whether to carry an umbrella if it is raining.
5. **Q5.** An iteration structure (e.g. a while loop or a for loop limited to 3 attempts) would repeatedly prompt the user to enter their PIN. Inside the loop, a selection structure (if-else) would check whether the entered PIN matches the correct one. If correct, the loop stops (e.g. using break) and access is granted; if incorrect, the loop continues until the maximum number of attempts is reached, after which the card would be locked. This shows how selection and iteration structures work together to control complex program behaviour.

3.5 Data Structures (Containers)

1. **Q1.** C
2. **Q2.** 0
3. **Q3.** The list/data must already be sorted.
4. **Q4.** Any two of: bubble sort, selection sort, insertion sort, quick sort.
5. **Q5.** Data structures allow related data to be grouped and organised logically under a single name, making programs easier to write, read and maintain, e.g. storing all of a class's marks in one list rather than using 40 separate variables. They also make operations such as searching and sorting more efficient and manageable, as illustrated by a teacher using a sorted list and binary search to quickly find a student's record among many.

3.6 Functions

1. **Q1.** B

2. **Q2.** A variable declared inside a function that can only be accessed within that function.
3. **Q3.** A value or variable that is passed into a function so it can be used within the function's body.
4. **Q4.** Any one of: makes code reusable, easier to test and debug, easier to maintain, reduces repetition, makes large programs more manageable.
5. **Q5.** User-defined functions allow a programmer to break a large task into smaller, self-contained blocks of code, each responsible for a specific task, e.g. separate functions to calculate tuition fees, transport fees, and total fees in a school fees system. This modular approach makes the program easier to read, test, and debug (since each function can be tested independently), promotes code reuse (since a function can be called multiple times from different parts of the program), and makes it easier to maintain or update the program in future, since changes to one function's logic do not require rewriting the entire program.

Appendix: Assessment Methods & Learning Resources

The following methods and resources may be used across all topics in this book to support learning and assessment, as recommended by the KICD Curriculum Design:

Category	Examples
Suggested Assessment Methods	Observation schedules, checklists, rating scales, rubrics, questionnaires, projects, journals, portfolios, oral questions, aural questions, interview schedules, learner's profile, written tests, anecdotal records.
Suggested Learning Resources	Reference materials (learners' books, teachers' guides, newspapers, journals, magazines); Audio resources (radio talks, audio tapes, DVDs/CDs); Visual resources (photos, charts, posters, graphs, flash cards); ICT resources (television, internet, computer hardware, productivity tools, visual programming tools, computer software); Community resources (field trips, computer science exhibitions/fairs, computer laboratories, computer science clubs); Human resources (teachers, computer laboratory technicians).
Suggested Non-Formal Activities	Educating community members on technology topics, demonstrating computer setup at social gatherings, debating on advantages/disadvantages of technologies, sharing experiences during computer science club activities.

FINAL NOTE TO THE LEARNER

Computer Science is a practical subject best learnt by doing. Wherever possible, practise the concepts in this book on a real computer - install an operating system in a virtual machine, write and run your own programs, and set up simple networks. Consistent practice, curiosity, and teamwork with your classmates will help you master this subject and open doors to exciting careers in technology.